

Low-pass filter

- Use the MATLAB `remez` function to design a low-pass filter with order 255 and transition band starting at 0.3 and ending at 0.4 of the sampling frequency.
- Apply input signal frequency of 2 KHz and maximum peak-to-peak voltage of 0.5 volts and observe filter output.
-

Matlab Code

```
% FIR low pass filter
% Design filter (normalized frequencies)
filterOrder = 255;
fpass = 0.3; % Passband edge (0.3 of Nyquist)
fstop = 0.4; % Stopband edge (0.4 of Nyquist)
b = firgr(filterOrder, [0 fpass fstop 1], [1 1 0 0], [1 10]);

% Frequency response analysis
[h, w] = freqz(b, 1, 4096); % More points for better resolution
freq = w/pi; % Normalized frequency

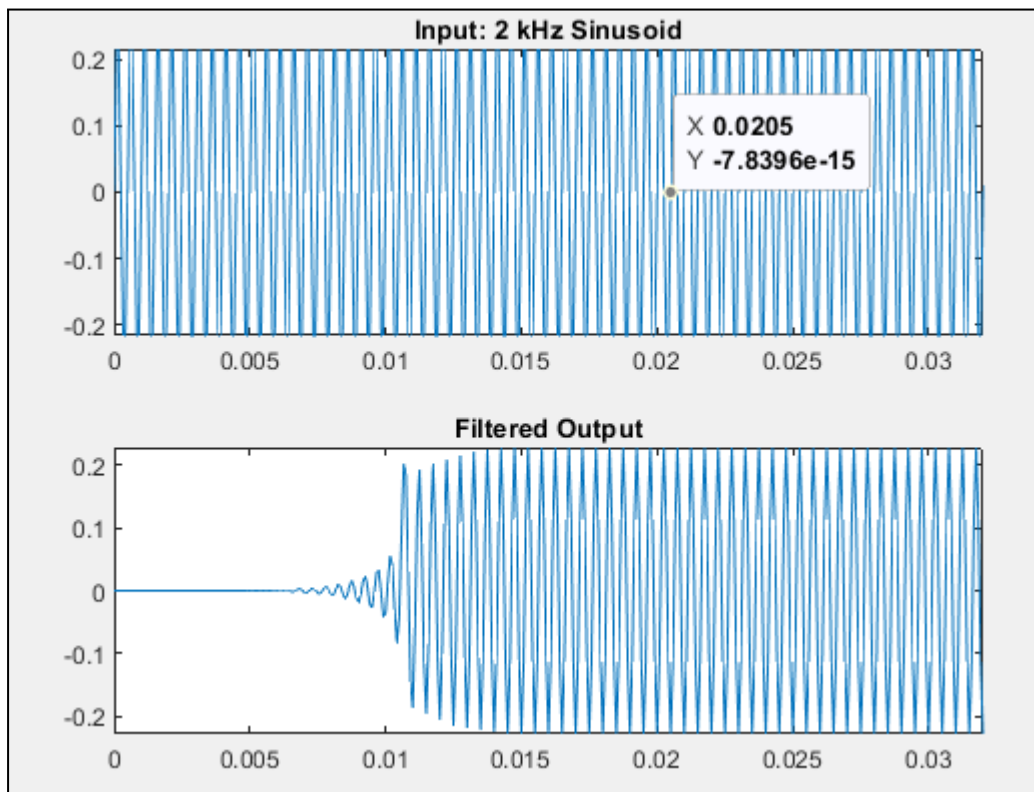
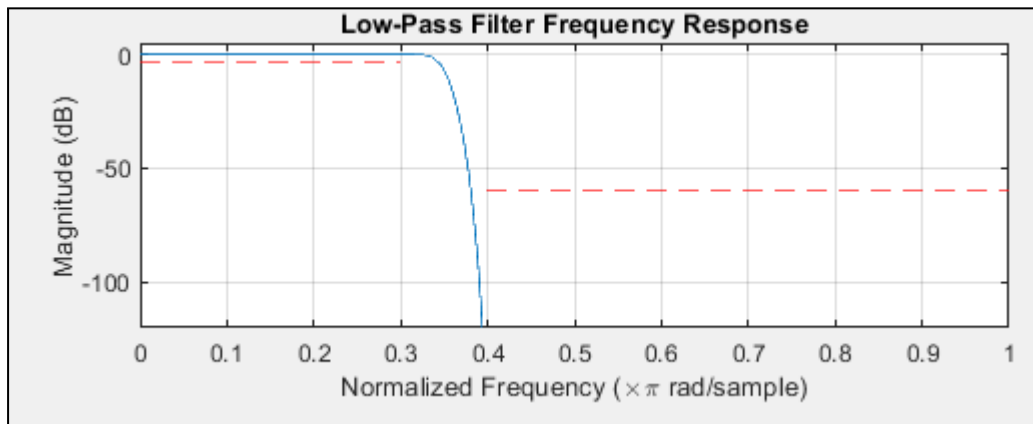
% Plot magnitude response
figure;
subplot(2,1,1);
plot(freq, 20*log10(abs(h)));
grid on;
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Magnitude (dB)');
title('Low-Pass Filter Frequency Response');
xlim([0 1]);
ylim([-120 5]); % Adjusted to clearly show stopband attenuation
line([0 fpass], [-3 -3], 'Color', 'red', 'LineStyle', '--');
line([fstop 1], [-60 -60], 'Color', 'red', 'LineStyle', '--');

% Generate input signal (2 kHz @ 8000 Hz sampling)
fs_actual = 12000;
t = 0:1/fs_actual:0.1; % 100 ms duration
input_signal = 0.25 * sin(2*pi*2000*t); % 0.5V peak-to-peak

% Apply filter
output_signal = filter(b, 1, input_signal);

% Plot results
figure;
subplot(2,1,1);
plot(t, input_signal); title('Input: 2 kHz Sinusoid'); xlim([0 0.032]);
subplot(2,1,2);
```

```
plot(t, output_signal); title('Filtered Output'); xlim([0 0.032]);
```



Comments:

- The output is incorrect until the filter's internal buffer fills with valid input samples. This is called the "step-up" transient.
- The transient lasts for filterOrder samples (255 samples). For $f_s = 8000$ Hz, this is:

$$\text{Transient} = \frac{255}{8000} \approx 32\text{msec}$$

- The cutoff is often approximated as the midpoint of the transition band:

$$fc \approx \frac{0.3+0.4}{2} = 0.35(\text{normalised to } fs)$$

- Passband edge (fpass): 0.3 (normalized frequency, corresponding to $0.3 \times fs/2$ Hz)
- Stopband edge (fstop): 0.4 (normalized frequency, corresponding to $0.4 \times fs/2$ Hz)

Exercise:

- Apply 2 KHz triangle (sawtooth), and square wave input signals with maximum peak-to-peak voltage of 0.5 volts. Observe the output of the filter.
- Repeat with a low-pass filter with order 255 and transition band starting at 0.1 and ending at 0.2 of the sampling frequency. Observe filter output for a 2 KHz input sinusoid, triangle (sawtooth), and square wave with maximum peak-to-peak voltage of 0.5 volts

Band-pass filter

- Design a band-pass filter with order 255 and first transition band starting and ending at 4/12 and 5/12 of sampling frequency and second transition band starting and ending at 7/12 and 8/12 of the sampling frequency.
- Apply input signal frequency of 2 KHz and maximum peak-to-peak voltage of 0.5 volts and observe filter output.

Matlab Code:

```
% Band Pass Filter
%Filter specifications
filterOrder = 255; % Order = 255 (256 taps)
fs = 1; % Normalized sampling frequency (1 = Nyquist)

% Normalized transition bands
f_low_stop = 4/12; % First stopband ends at 4/12
f_low_pass = 5/12; % First passband starts at 5/12
f_high_pass = 7/12; % Second passband ends at 7/12
f_high_stop = 8/12; % Second stopband starts at 8/12

% Band edges for firgr (must be in increasing order)
f_edges = [0, f_low_stop, f_low_pass, f_high_pass, f_high_stop, 1];
a_desired = [0, 0, 1, 1, 0, 0]; % Desired amplitudes in each band
```

```

% Design filter with weights (higher weight = stricter tolerance)
weights = [1, 10, 1]; % Weights for [stopband1, passband, stopband2]
b = firgr(filterOrder, f_edges, a_desired, weights);

% Frequency response analysis
[h, w] = freqz(b, 1, 4096);
freq = w / pi; % Normalized frequency (0 to 1)

% Plot magnitude response
figure;
plot(freq, 20*log10(abs(h)));
grid on;
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Magnitude (dB)');
title('Band-Pass Filter Frequency Response');
xlim([0 1]);
ylim([-100 5]);
hold on;
line([f_low_pass, f_low_pass], [-100, 5], 'Color', 'red', 'LineStyle', '--');
line([f_high_pass, f_high_pass], [-100, 5], 'Color', 'red', 'LineStyle', '--');

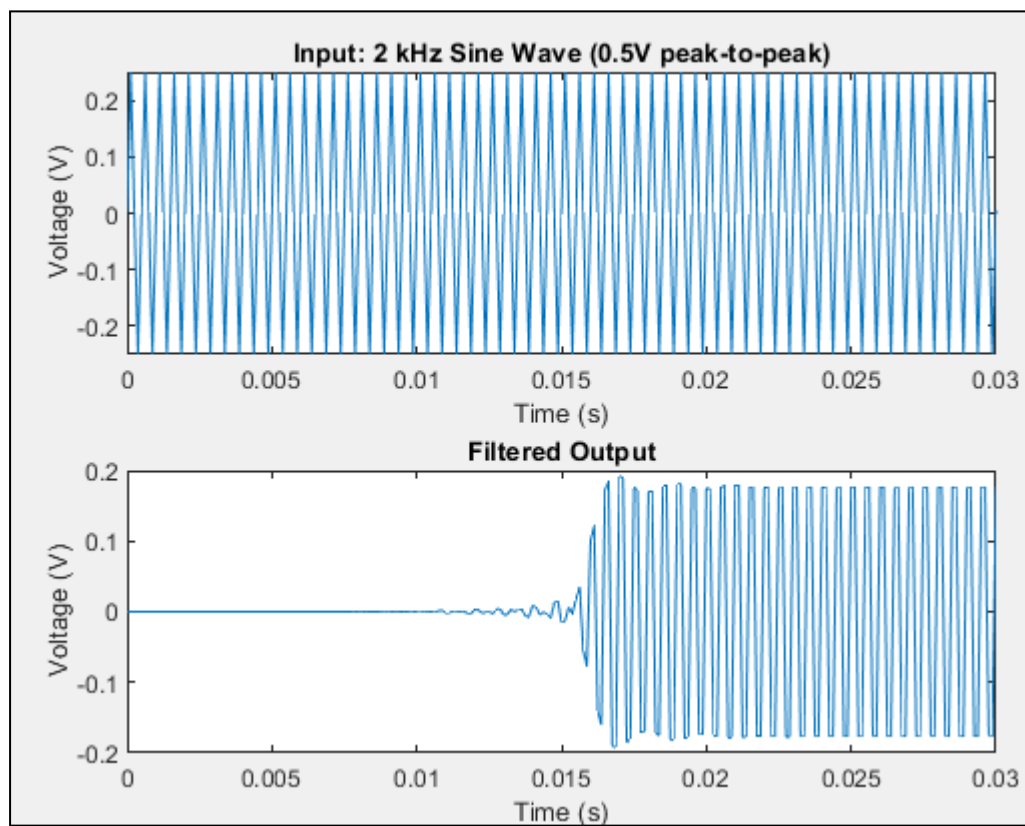
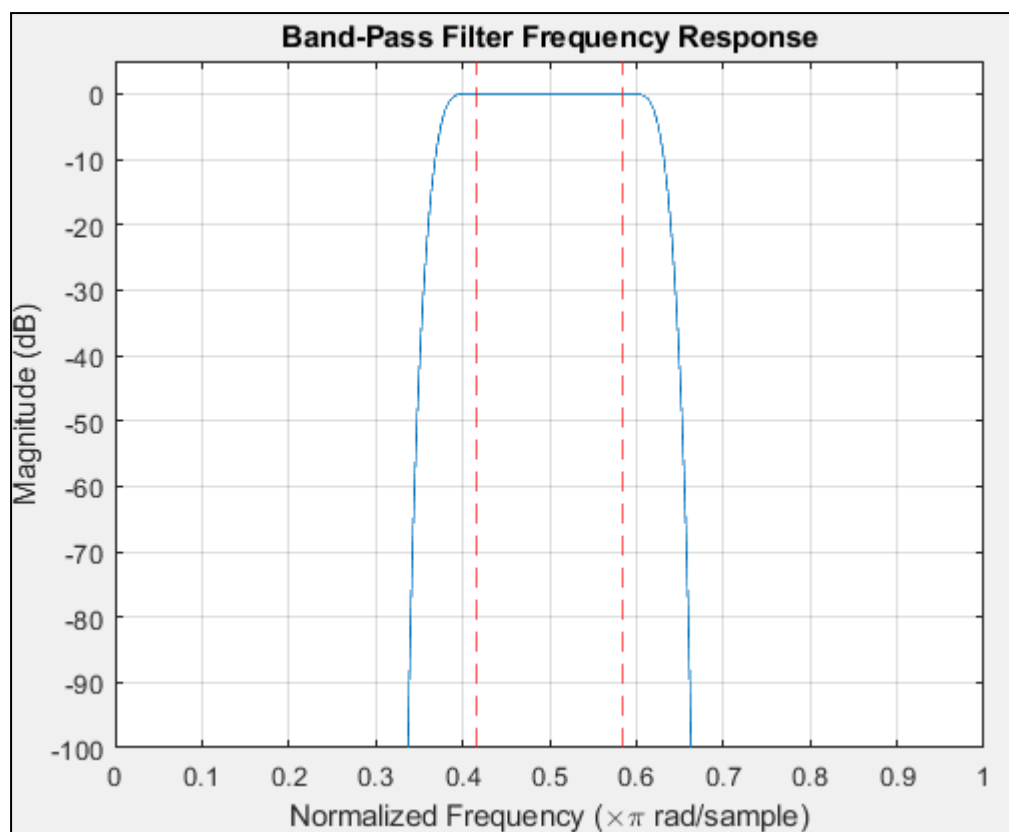
fs = 8000; % Sampling frequency (Hz). Must be >2*2000 Hz (Nyquist)
% Input signal (2 kHz, 0.5V peak-to-peak)
duration = 0.1; % 100 ms simulation
t = 0:1/fs:duration; % Time vector
input_signal = 0.25 * sin(2*pi*2000*t); % Amplitude = 0.25V (0.5V peak-to-peak)

output_signal = filter(b, 1, input_signal); % Filter the input

figure;
subplot(2,1,1);
plot(t, input_signal);
title('Input: 2 kHz Sine Wave (0.5V peak-to-peak)');
xlabel('Time (s)');
ylabel('Voltage (V)');
xlim([0, 0.03]); % Show first 30 ms

subplot(2,1,2);
plot(t, output_signal);
title('Filtered Output');
xlabel('Time (s)');
ylabel('Voltage (V)');
xlim([0, 0.03]);

```



Comments:

- transient response for filter : (first filterOrder/2 samples) delay = filterOrder/2; Group delay = 127.5 samples. Phase delay = 127.5 samples (15.94 ms at fs = 8000 Hz).
- For fs = 8000 Hz:
 - Passband edges: Lower: $5/12 * 4000 = 1666.67$ Hz
 - Upper: $7/12 * 4000 = 2333.33$ Hz2 kHz falls within the passband ($1666.67 < 2000 < 2333.33$), so: output amplitude $\approx$ 0.25V (same as input, minus minor ripple).

Exercise:

- Apply 2 KHz triangle (sawtooth), and square wave input signals with maximum peak-to-peak voltage of 0.5 volts. Observe the output of the filter.
- Repeat with a low-pass filter with order 255 and transition band starting at 0.1 and ending at 0.2 of the sampling frequency. Observe filter output for a 2 KHz input sinusoid, triangle (sawtooth), and square wave with maximum peak-to-peak voltage of 0.5 volts

Third-order Harmonics Exercise

Design and implement a filter that will pass only the third harmonic of a 1kHz square wave (i.e. the harmonic at 3kHz). Justify your choices for the sampling rate, filter order, pass band, transition band, and stop band. Include all relevant Matlab plots and data from tests.